Enhancement of Mutation Testing via Fuzzy Clustering and Multi-population Genetic Algorithm

Xiangying Dang, Dunwei Gong*, Xiangjuan Yao*, Tian Tian, and Huai Liu

Abstract—Mutation testing, a fundamental software testing technique, which is a typical way to evaluate the adequacy of a test suite. In mutation testing, a set of mutants are generated by seeding the different classes of faults into a program under test. Test data shall be generated in the way that as many mutants can be killed as possible. Thanks to numerous tools to implement mutation testing for different languages, a huge amount of mutants are normally generated even for small-sized programs. However, a large number of mutants not only leads to a high cost of mutation testing, but also make the corresponding test data generation a non-trivial task. In this paper, we make use of intelligent technologies to improve the effectiveness and efficiency of mutation testing from two perspectives. A machine learning technique, namely fuzzy clustering, is applied to categorize mutants into different clusters. Then, a multi-population genetic algorithm via individual sharing is employed to generate test data for killing the mutants in different clusters in parallel when the problem of test data generation as an optimization one. A comprehensive framework, termed as **FUZGENMUT**, is thus developed to implement the proposed techniques. The experiments based on nine programs of various sizes show that fuzzy clustering can help to reduce the cost of mutation testing effectively, and that the multi-population genetic algorithm improves the efficiency of test data generation while delivering the high mutant-killing capability. The results clearly indicate that the huge potential of using intelligent technologies to enhance the efficacy and thus the practicality of mutation testing.

Index Terms—mutation testing, fuzzy clustering, mutation clustering, test data generation, multi-population genetic algorithm (MGA)

1 Introduction

Software testing, one vital process in the software development life cycle, aims at software quality assurance by detecting as many software faults as possible. Basically speaking, in software testing, some program inputs are generated as test data, which will be used to execute the software under test [1]. Various techniques have been proposed to guide test data generation in detecting different classes of faults.

Mutation testing, proposed by Hamlet et al. [2] and DeMillo et al. [3], is a typical fault-based testing technique. Mutation testing makes use of some so-called *mutation operators* [1] to apply syntactic changes to a program under test. A set of faulty variants, termed as *mutants*, are generated. Test data can then be generated with the purpose of "*killing*" as many mutants as possible, showing a high potential of effectively detecting a variety of faults [1], [4]. There are two main criteria to check whether a mutant has been killed by a test datum, namely the *strong mutation* and the *weak mutation*, based on the conditions of reachability, infection, and propagation [1]. The weak mutation refers to the situation where the test datum has executed the mutated statement (reachability) and triggered an incorrect state of the program (infection). The strong mutation further requires that the incorrect state be propagated to the output of the program (propagation) such that the mutant's output is different from that of the program under test. On one hand, the strong mutation is considered to be more accurate than the weak mutation in reflecting the real software failures, which can only be guaranteed through propagation. On the other hand, right due to the lack of propagation, the execution of weak mutation can be much more efficient than that of strong mutation.

To provide an unbiased simulation of the wide variety of faults in real life, a large number of mutants are normally required in mutation testing. Numerous tools have been developed to automatically construct a huge mount of mutants for different programming languages [4], [5]. Nevertheless, a large number of mutants incur a high computation overhead of executing mutation testing. It is also a non-trivial task to generate test data that can kill most of the mutants. Therefore, lots of studies have been conducted to improve the efficiency of mutation testing and thus its applicability,

- ∗ *Corresponding author: D. Gong; X. Yao.*
- *X. Dang is with School of Information and Control Engineering, China University of Mining and Technology, Xuzhou, 221116, China; she is also with School of Information Engineering (School of Big Data), Xuzhou University of Technology, and Key Laboratory of Intelligent Industrial Control Technology of Jiangsu Province, Xuzhou, 221000, China (e-mail: dangpaper@163.com).*
- *D. Gong is with School of Information and Control Engineering, China University of Mining and Technology, Xuzhou, 221116, China; he is also with School of Information Science and Technology, Qingdao University of Science and Technology, Qingdao, Shandong, 266061, China (e-mail: dwgong@vip.163.com).*
- *X. Yao is with School of Mathematics, China University of Mining and Technology, Xuzhou, 221116, China (e-mail: yaoxj@cumt.edu.cn).*
- *T. Tian is with School of Computer Science and Technology, Shandong Jianzhu University, Jinan 250101, China (e-mail: tian_tiantian@126.com).*
- *H. Liu is with the Department of Computer Science and Software Engineering, Swinburne University of Technology, Hawthorn VIC 3122, Australia (e-mail: hliu@swin.edu.au).*

Manuscript received August 1, 201X; revised August 1, 201X, September 22,201X,and December 4,20XX; accepted December 15,20XX.

which is also the ultimate goal of the present study.

Numerous strategies have been proposed to reduce the cost of mutation testing [1], [6]. In a survey, Jia and Harman [5] classified the cost reduction for mutation testing into two categories. One major category of such strategies aims at decreasing the number of mutants, such as high order mutation [7], mutant sampling [8], [9], and selective mutation [10]. Another category is focused on reducing the execution cost [1], [5], [11], such as *sorting* and *clustering* of mutants [12], [13]. The objective of sorting mutants is to give higher priorities to those mutants that are regarded as more important according to various testing requirements [14], [15], such as mutation operators and killed difficulties. Mutant clustering refers to the arrangement of mutants into different clusters according to certain principles [16], [17]. Papadakis et al. [18] pointed out that mutant classification is beneficial to balancing effectiveness and efficiency for mutation testing. Mutant clustering, proposed by Hussain [16], is similar to mutant classification. All the mutants in the same cluster are expected to have a high opportunity to be killed by a set of similar test data. Ji et al. [19] extended the work of mutant clustering, where a domain reduction technique was employed to avoid the execution of all mutants. Empirical studies [16], [17], [19] showed that mutant clustering can help maintain a high mutant killing capability by just utilizing a small number of mutants, especially those in the "center" of clusters.

Another aspect for improving the efficiency of mutation testing is to effectively generate test data [4], [20], [21], [22], [23], for which search-based techniques [24], [25] have attracted lots of interest. As one major search-based method, *genetic algorithm* (*GA*) is a typical global optimization algorithm inspired by the mechanism of biological evolution. When generating test data, a population composed of some individuals continuously searches optimal solutions in the input domain under the guidance of a specific fitness function [26], [27]. GAs have been used to generate test data in the context of mutation testing [24].

In this paper, we attempt to enhance the efficacy of mutation testing from both of the above two aspects. On one hand, we make use of a combination of sorting and clustering to reduce execution cost of test data generation for killing the mutants. Mutants are first sorted based on their *killed difficulties* (that is, how difficult they are killed). After that, they are further categorized into different clusters according to the *similarity* among them (which refers to the homogeneity of test data that kill them). To guarantee low cost of executing mutants, the weak mutation is used in calculating the killed difficulty and similarity. Within each cluster, the mutant with the highest killed difficult will be selected as the "center" because test data killing those "hard-to-kill" mutants normally also have a very high probability to kill other relatively "easy-to-kill" mutants. If test data that kill cluster centers are preferentially generated, it is enough for them to execute these mutants in these clusters instead of all the mutants, leading to reduce the cost in executing mutants. In this way, for a large number of mutants, clustering mutants is employed to reduce the cost in generating test data that execute and kill these mutants. In addition, most studies on clustering mutants [16], [17], [19] have adopted "hard clustering", that is, a mutant belongs to at most one cluster, which does not accord with real-life cases — some mutants could show a certain degree of similarity to different clusters' centers, and thus might be allocated to multiple clusters at the same time. In contrast, it is rational to say that a mutant belongs to a cluster at a certain degree. In this circumstance, a mutant can be assigned to more than one cluster. To this end, we adopt fuzzy clustering [28], [29], [30], which is a typical machine learning technique, when clustering mutants.

On the other hand, we apply the *multi-population genetic algorithm* (*MGA*) to further improve the efficiency of test data generation for mutation testing. Given that all mutants are divided into multiple clusters, the problem of generating test data that kill these mutants can be divided into several sub-problems. In this circumstance, we expect to seek an algorithm to solve all these sub-problems simultaneously, rather than to solve them in sequence. As compared with a *single-population genetic algorithms* (*SGA*) with only one population, MGA generally has superior performance [31]. In addition, since fuzzy clustering makes it possible for a mutant to belong to multiple clusters, it is more appealing to adopt MGA to generate test data, which is our key technique for efficiently generating test data. The nature of parallelism for MGA means that multiple test data for killing different mutants can be generated simultaneously. In MGA, a sub-population is evolved to generate test data to kill all the mutants in the same cluster, while there exists an individual shared among different sub-populations that are evolved in parallel. In this way, MGA can reduce the cost of generating test data to kill the mutants. In order to guarantee the accuracy of fault detection, we utilize the strong mutation to generate test data.

Based on all the above techniques, especially for the fuzzy clustering and MGA, we develop a comprehensive framework, namely **FUZGENMUT**, for generating test data to kill a large number of mutants. This paper has the following three contributions:

- It significantly reduces the cost of mutation testing, mainly through sorting and fuzzy clustering under the weak mutation criterion, and thus improves the practicality of mutation testing.
- The new **FUZGENMUT** framework is able to quickly generate test data via *MGA via individual sharing*, while delivering a high mutant-killing capability; in other words, it substantially enhances the efficiency of test data generation.
- A comprehensive empirical study demonstrates that the use of intelligent technologies can largely improve the effectiveness and efficiency of mutation testing from different perspectives.

The paper is organized as follows. Section 2 introduces the background information for our work. The **FUZGENMUT** framework is described in Section 3, together with the related concepts and detailed algorithms. In Section 4, the design and settings of the empirical studies are presented. The experimental results are summarized and analyzed in Section 5. Threats to validity are discussed in Section 6. In Section 7, we present the studies related to our work. In Section 8, we conclude the paper and highlight some future work.

2 BACKGROUND

2.1 Mutation testing

For mutation testing, faults are generally seeded into a program under test deliberately by a number of simple syntactic changes to create a set of faulty programs called mutants. The syntactic change rules are called as mutant operators [15], [32], [33]. If a test datum causes a mutant to show a different execution behavior from the program under test, it is said that the test datum kills the mutant, which implies the detection of the corresponding fault. In mutation testing, test data can be generated with the purpose of "killing" as many mutants as possible. If only one fault is seeded into one statement, the resultant mutant is called a first-order mutant, and the corresponding technique is called the first-order mutation testing [5], [34].

Papadakis et al. [35] have proposed the method of automatically generating test data based on weak mutation testing. Like many other mutation testing methods, this method makes use of a "mutant schemata" technique to compose all mutants (which are normally first-order) into one meta-program that only requires one compilation. Note that the weak mutation only requires the checking of the change in the program state (not the final program output) once the mutated statement has been reached. Therefore, a so-called "*mutant branch*" is particularly used to represent a mutant [10], where a mutant will be considered as killed if the mutant branch confirms that the program state is changed at the mutated statement. Taking the triangular classification program as an example, which is provided in Fig. 1 (a). A mutant can be generated by changing the original statement, "$if(x > z)$", to the mutation statement, "$if(x == z)$" (shown in Fig. 1(b)). A similar process can be followed to create more mutants and construct corresponding mutant branches, such as those in Fig. 1(c).

In mutation testing, there exist some mutants that cannot be killed by any test datum. They are called equivalent mutants [36], [37]. Theoretically speaking, though an equivalent mutant has some syntactic differences from the program under test, they are semantically identical to each other and thus always have the same execution behaviors.

The most fundamental and commonly used metric in mutation testing is the mutation score, which refers to the ratio of the number of killed mutants to the number of all non-equivalent mutants [1], [5], represented as:

$$MS = \frac{\text{No. of the killed mutants}}{\text{No.of the non-equivalent mutants}} \quad (1)$$

In essence, the mutation score denotes the degree of achievement of a test suite in detecting various types of faults. Thus, the mutation score has been widely used as the adequacy metric of a test suite.

2.2 Clustering

Clustering is a task to divide data into different groups, normally based on some similarity characteristics. It has been widely used in various areas, including pattern recognition and classification. In particular, clustering can be considered as a form of unsupervised classification [38]. Naively speaking, the members within a cluster should be similar to one another, while different from those in other clusters.

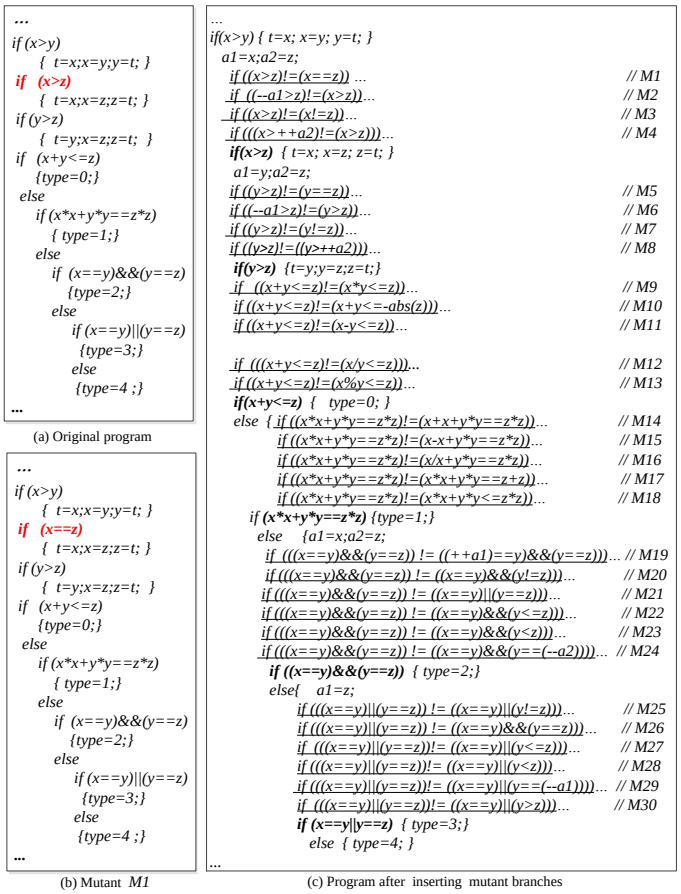


Fig. 1. Triangular classification program

There exist different classes of clustering algorithms [39]. One main category of clustering algorithms is the hierarchical clustering method, such as agglomerative algorithm and divisive algorithm. Another major category is the centroid-based algorithm, such as K-means algorithm and fuzzy c-means algorithm.

One key challenge in clustering is to handle cluster overlapping, where some data naturally show certain degrees of similarity to different clusters. Many clustering techniques are hard (or crisp) clustering, where a data point belongs to one and only one cluster. To address this issue, fuzzy clustering [28], [29] allows a data point to reside in multiple clusters. Fuzzy clustering has been applied in many application domains. For example, news can be correlated to different topics, including science, business, and even sports. The text categorization based on fuzzy clustering can allocate the news into different corresponding clusters, instead of just one.

There are also some metrics (or called quality indices) to reflect the performance of clustering [39], such as the compactness (CP), the separation (SP), and the in-group proportion (IGP), which address various clustering problems from different perspectives.

2.3 Genetic algorithms

Genetic algorithms (GAs) are robust search and optimization techniques, which are developed based on the ideas and concepts from genetic and evolutionary theory [26], [27]. They provide an alternative to traditional optimization techniques by using directed random searches to locate optimal solutions in complex landscapes. GAs show unique advantages and efficiency in solving problems with high complexity, such as the problems with large space, multi-peak, and nonlinearity.

Historically, SGA is characterized as an evolutionary scheme performed on a single population of individuals. MGA [27] is an extension of SGA by dividing a population into several isolated sub-populations, within which the evolution proceeds and individuals are allowed to migrate from one sub-population to another. This model is analogous to that used in parallel island GAs, without emphasizing the parallelism issue. Compared with the SGA, the MGA is more similar to the natural genetic evolution.

Recently, GAs have become one popular way to support the automatic test data generation and some encouraging results have been observed [31], [40], [41]. Yao et al. [27] have proposed a method of generating test data simultaneously via the *MGA via individual sharing* for multiple paths. In the algorithms, each sub-population optimizes one subproblem, and all sub-populations evolve in parallel. A key process is the individual sharing of different sub-populations. More specifically, every time when the evolutionary operations of a generation finish, the algorithm not only determines whether or not an individual is an optimal solution of the sub-population it belongs to, but also does the same for other sub-populations. In this way, the efficiency of finding optimal solutions for each subproblem can be improved, while the overall complexity of the algorithm is not increased significantly.

3 Proposed Approach

We propose a new framework, namely **FUZGENMUT**, which makes use of *FUZ*zy clustering and multi-population *GEN*etic algorithm, to improve the efficiency of test data generation in *MUT*ation testing.

In this paper, **FUZGENMUT** is implemented through six major steps, as shown in Fig. 2. In **Step 1**, a meta-program is composed following the previous study [10], [35], which is commonly suitable for most application scenarios of mutation testing.

From **Steps** 2 to 6, **FUZGENMUT** adopts the method of divide and conquer, mainly through the intelligent technologies of fuzzy clustering (**Steps 2** to **4**) and MGA (**Steps 5** and **6**). Fuzzy clustering facilitates the "division" of mutants into multiple clusters. MGA is then employed to "conquer" these clusters, that is, to generate test data for killing the mutants in parallel.

Note that when calculating the killed difficulty and the similarity of the mutants (**Steps 2**), sorting and clustering them (**Steps 3** and **4**), we assume the existence of test data that kill mutants, which are generated using the random method under the criterion of weak mutation testing. Compared with weak mutation testing, we take more care of strong mutation testing, and expect to generate high quality test data to kill mutants under the criterion.

A mathematical model should be formulated (**Step 5**) before using MGA to generate test data for killing the mutants (**Step 6**). In this study, we take the program input vector as the decision vector, and formulate the objective function based on a flag indicating whether a mutant is killed or not. Due to the difficulty of such an objective in guiding the search, we add a constraint that reflects the capability of program input in covering the mutated statement. In this way, the problem of generating test data that kill a mutant can be formulated as an optimization problem with one objective and the unique constraint.

Based on the above formulation, we design a fitness function by the penalty function method, which is a typical way of tackling constrained optimization problems. The ultimate outcome of **FUZGENMUT** is a test suite that are effective in detecting a variety of faults, so we apply the strong mutation criterion in **Step 6** (unlike the weak mutation in **Step 2**). In this way, the MGA-generated test data are capable of revealing observable software failures, which are normally reflected by the different outputs between a mutant and the program under test.

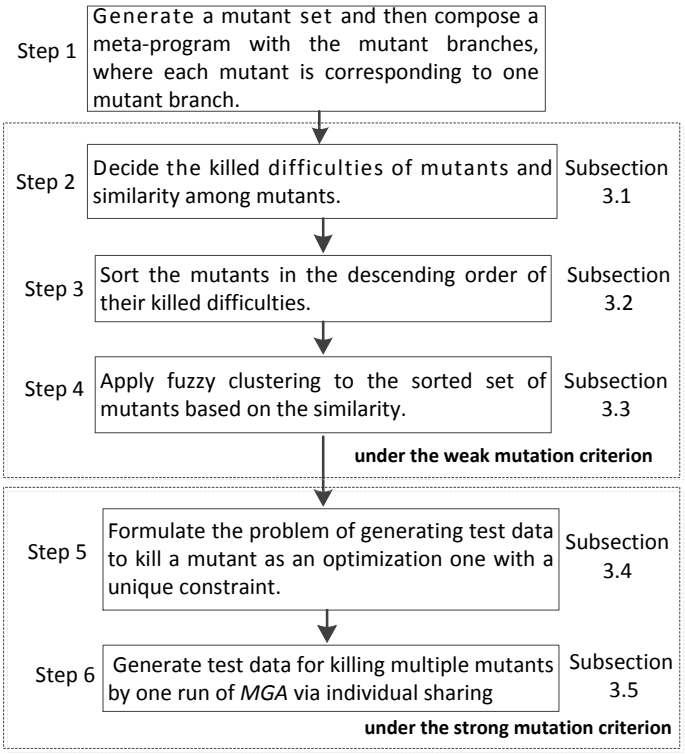


Fig. 2. The application scenarios of **FUZGENMUT**

3.1 Basic Definitions

In this section, the basic definitions of "killed difficulty" of each mutant and the "similarity" between mutants (in **Step 2**) are given as follows.

3.1.1 Killed difficulty of a mutant

Papadakis et al. [42] and Yao et al. [33] claimed that a hard-to-kill mutant is killed by only a small number of test data in

a test suite. Thus we employ test data to evaluate the killed difficulty of a mutant based on mathematical statistics in this paper [43]. To reflect the possibility of a test datum, X, killing mutant M_i, the following random variable is defined:

$$\mu_i(X) = \begin{cases} 1 \ X \ kills \ M_i \\ 0 \ otherwise \end{cases}. \quad (2)$$

Eq. (2) shows that $\mu_i(X) \sim b(1, p_i)$, where p_i is the probability of $\mu_i(X)$ = 1.

Note that the input domain (the set of all possible inputs) for a program is normally extremely large, if not infinite. Therefore, in practice, we approximate the value of p_i based on a sample suite of test data. Suppose that there are R sample test data, denoted by $X_1, X_2, \cdots, X_R$. For a non-equivalent mutant M_i, its killed difficulty is defined as follows:

***Definition* 1:** The *killed difficulty* of M_i, denoted by $Dif(M_i)$, can be defined as:

$$Dif(M_i) = 1 - \frac{\sum_{k=1}^{R} \mu_i(X_k)}{R}. \quad (3)$$

Eq. (3) indicates that $0 \leq Dif(M_i) \leq 1$, and the greater the value of $Dif(M_i)$ is, the more difficult M_i is to be killed.

Suppose that for M_1 in Fig. 1(c), 170 out of 500 test data (that is, $R = 500$) can kill it; in other words, $\sum_{k=1}^{R} \mu_1(X_k)$=170. Then, the *killed difficulty* of M_1 is $Dif(M_1) = 1 - \frac{170}{500} \approx 0.66$.

3.1.2 Similarity between mutants

Basically speaking, we estimate the similarity between two mutants based on the number of common test data that kill them. Suppose that $M_i, M_j (i, j = 1, 2, ..., n, i \neq j$, where n refers to the total number of mutants) are two mutants. We can obtain two random variables, $\mu_i(X)$ and $\mu_j(X)$, based on Eq. (2).

The probability of $\mu_j(X)$ being 1 when $\mu_i(X)$ =1 is represented as:

$$P\{\mu_j(X) = 1 | \mu_i(X) = 1\} = \frac{P\{\mu_j(X) = 1, \mu_i(X) = 1\}}{P\{\mu_i(X) = 1\}}. \quad (4)$$

For R sample data, $X_1, X_2, \cdots, X_R$, $P\{\mu_j(X_k) = 1 | \mu_i(X_k) = 1\}$ can be approximated as:

$$P\{\mu_j(X_k) = 1 | \mu_i(X_k) = 1\} \approx \frac{\sum_{k=1}^{R} \mu_i(X_k)\mu_j(X_k)}{\sum_{k=1}^{R} \mu_i(X_k)}. \quad (5)$$

Eq. (5) implies that if there is a strong correlation between $\mu_i(X_k)$ and $\mu_j(X_k)$, then the value of $P\{\mu_j(X_k) = 1 | \mu_i(X_k) = 1\}$ will be very high, approaching 1.

Given the fact that the conditional probabilities of $\mu_i(X_k)$ and $\mu_j(X_k)$ can reflect the correlation between M_i and M_j, we have the following definition of the similarity between two mutants.

***Definition* 2:** The *similarity* between M_i and M_j, denoted as $\alpha_{i,j}$, can be defined as:

$$\alpha_{i,j} = \frac{\sum_{k=1}^{R} \mu_i(X_k)\mu_j(X_k)}{\sum_{k=1}^{R} \mu_i(X_k)}. \quad (6)$$

In other words, $\alpha_{i,j}$ is actually the conditional probability $P\{\mu_j(X_k) = 1 | \mu_i(X_k) = 1\}$. From Eq. (6), it is obvious that $\alpha_{i,j} \in [0, 1]$. The *similarity* between M_i and M_j is the strongest when $\alpha_{i,j} = 1 (i \neq j)$. On this circumstance, all test data that kill M_i can also kill M_j. In contrast, the *similarity* between M_i and M_j will be the weakest if $\alpha_{i,j} = 0$. Generally speaking, the larger the value of $\alpha_{i,j}$ is, the more similar M_j is to M_i. Note that $\alpha_{i,j}$ is not necessarily equal to $\alpha_{j,i}$.

After calculating the similarity between each pair of mutants using Eq. (6), we can finally get a so-called *similarity matrix*, denoted as Λ:

$$\Lambda = \begin{bmatrix} \alpha_{1,1} \ \alpha_{1,2} \ .... \ \alpha_{1,j} \ .... \ \alpha_{1,n} \\ \alpha_{2,1} \ \alpha_{2,2} \ .... \ \alpha_{2,j} \ .... \ \alpha_{2,n} \\ .... \quad .... \quad .... \\ \alpha_{i,1} \ \alpha_{i,2} \ .... \ \alpha_{i,j} \ .... \ \alpha_{i,n} \\ .... \quad .... \quad .... \\ \alpha_{n,1} \ \alpha_{n,2} \ .... \ \alpha_{n,j} \ .... \ \alpha_{n,n} \end{bmatrix}$$

Fig. 3 gives a similarity matrix for the 30 mutants in Fig. 1(c). Suppose that among the 170 test data that can kill M_1, there are 7 and 170 test data that can kill M_2 and M_5, respectively. According to Eq. (6), we can get $\alpha_{1,2} = \frac{7}{170} \approx 0.04$ and $\alpha_{1,5} = 1$. The values imply that M_5 is more similar to M_1 than M_2.

Note that, we employed an initial suite of sample test data as the basis for the calculation of the killed difficulty and similarity. The performance of this test suite must also be considered. More details about the initial test suite will be given in Section 4.2.3.

3.2 Sorting mutants

In **Step 3** of **FUZGENMUT**, all mutants in $M = \{M_1, M_2, ..., M_n\}$ shall be sorted in the descending order of the killed difficulty. The result will be a sorted set of the mutants, denoted as $M^S = \{M_1^S, M_2^S, \cdots, M_n^S\}$.

Table 1 gives an exemplary M^S for the 30 mutants in Fig. 1, where $M_1^S = M_{14}$, $M_2^S = M_{16}, \ldots$.

3.3 Fuzzy clustering of the sorted mutants

Algorithm 1 gives the procedure about the fuzzy clustering of mutants. As can been seen from lines 3 to 7 that we always select the top one, M_1^S, from M^S as the "center" of a cluster. Following that, M_1^S is deleted from M^S and M because M_1^S is not arranged to other clusters.

After the determination of the center, one cluster will be expanded by adding other cluster members from M, i.e., "non-center" mutants (lines 9 to 15). In Algorithm 1, we judge whether mutant $M_j \in M$ belongs to C_k according to similarity $\alpha_{i,j}$ between M_i and M_j , where M_i is a center

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TSE.2021.3052987, IEEE Transactions on Software Engineering

$\Lambda=$

1.00	0.04	0.00	0.04	1.00	0.05	0.00	0.05	0.42	0.47	0.53	0.53	0.53	0.00	0.02	0.00	0.00	0.30	0.00	0.02	0.04	0.02	0.02	0.00	0.50	0.04	0.49	0.50	0.06	0.02
1.00	1.00	0.00	1.00	1.00	0.03	0.00	0.03	0.28	0.41	0.59	0.59	0.59	0.00	0.01	0.00	0.00	0.25	0.01	0.00	0.01	0.00	0.00	0.00	0.59	0.01	0.57	0.59	0.04	0.01
0.00	0.00	1.00	0.00	0.52	0.02	0.48	0.02	0.42	0.47	0.53	0.53	0.53	0.00	0.02	0.00	0.00	0.29	0.00	0.01	0.03	0.01	0.01	0.00	0.52	0.03	0.50	0.52	0.07	0.02
1.00	1.00	0.00	1.00	1.00	0.03	0.00	0.03	0.28	0.41	0.59	0.59	0.59	0.00	0.01	0.00	0.00	0.25	0.01	0.00	0.01	0.00	0.00	0.00	0.59	0.01	0.57	0.59	0.04	0.01
0.50	0.02	0.50	0.02	1.00	0.05	0.00	0.05	0.42	0.46	0.54	0.54	0.54	0.00	0.03	0.00	0.00	0.30	0.00	0.01	0.04	0.01	0.01	0.00	0.52	0.04	0.49	0.52	0.08	0.03
0.51	0.01	0.49	0.01	1.00	1.00	0.00	1.00	0.00	0.00	1.00	1.00	1.00	0.00	0.00	0.00	0.00	0.19	0.00	0.02	0.02	0.02	0.02	0.02	0.98	0.02	0.98	0.98	0.98	0.00
0.00	0.00	1.00	0.00	0.00	0.00	1.00	0.00	0.43	0.49	0.51	0.51	0.51	0.00	0.00	0.00	0.00	0.28	0.00	0.02	0.02	0.02	0.02	0.00	0.50	0.02	0.50	0.50	0.04	0.00
0.51	0.01	0.49	0.01	1.00	1.00	0.00	1.00	0.00	0.00	1.00	1.00	1.00	0.00	0.00	0.00	0.00	0.19	0.00	0.02	0.02	0.02	0.02	0.02	0.98	0.02	0.98	0.98	0.98	0.00
0.34	0.01	0.66	0.01	0.68	0.00	0.32	0.00	1.00	1.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
0.34	0.01	0.66	0.01	0.67	0.00	0.33	0.00	0.89	1.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
0.34	0.02	0.66	0.02	0.69	0.06	0.31	0.06	0.00	0.00	1.00	1.00	1.00	0.00	0.04	0.00	0.00	0.55	0.00	0.02	0.06	0.02	0.02	0.00	0.98	0.06	0.94	0.98	0.12	0.04
0.34	0.02	0.66	0.02	0.69	0.06	0.31	0.06	0.00	0.00	1.00	1.00	1.00	0.00	0.04	0.00	0.00	0.55	0.00	0.02	0.06	0.02	0.02	0.00	0.98	0.06	0.94	0.98	0.12	0.04
0.34	0.02	0.66	0.02	0.69	0.06	0.31	0.06	0.00	0.00	1.00	1.00	1.00	0.00	0.04	0.00	0.00	0.55	0.00	0.02	0.06	0.02	0.02	0.00	0.98	0.06	0.94	0.98	0.12	0.04
0.00	0.00	1.00	0.00	1.00	0.00	0.00	0.00	0.00	0.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
0.27	0.01	0.73	0.01	1.00	0.00	0.00	0.00	0.00	0.00	1.00	1.00	1.00	0.01	1.00	0.01	0.01	0.00	0.05	0.00	0.99	0.00	0.00	0.00	0.99	0.99	0.00	0.99	0.99	0.99
0.00	0.00	1.00	0.00	1.00	0.00	0.00	0.00	0.00	0.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
0.00	0.00	1.00	0.00	1.00	0.00	0.00	0.00	0.00	0.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
0.35	0.01	0.65	0.01	0.70	0.02	0.30	0.02	0.00	0.00	1.00	1.00	1.00	0.00	0.00	0.00	0.00	1.00	0.00	0.02	0.02	0.02	0.02	0.00	0.98	0.02	0.98	0.98	0.03	0.00
0.20	0.20	0.80	0.20	1.00	0.00	0.00	0.00	0.00	0.00	1.00	1.00	1.00	0.00	1.00	0.00	0.00	0.00	1.00	0.00	1.00	0.00	0.00	0.00	1.00	1.00	0.00	1.00	1.00	1.00
0.63	0.00	0.37	0.00	0.63	0.05	0.37	0.05	0.00	0.00	1.00	1.00	1.00	0.00	0.00	0.00	0.00	0.48	0.00	1.00	1.00	1.00	1.00	0.08	0.00	1.00	0.00	0.00	0.00	0.00
0.41	0.01	0.59	0.01	0.86	0.02	0.14	0.02	0.00	0.00	1.00	1.00	1.00	0.00	0.62	0.00	0.00	0.18	0.03	0.38	1.00	0.38	0.38	0.03	0.62	1.00	0.00	0.62	0.62	0.62
0.63	0.00	0.37	0.00	0.63	0.05	0.37	0.05	0.00	0.00	1.00	1.00	1.00	0.00	0.00	0.00	0.00	0.48	0.00	1.00	1.00	1.00	1.00	0.08	0.00	1.00	0.00	0.00	0.00	0.00
0.63	0.00	0.37	0.00	0.63	0.05	0.37	0.05	0.00	0.00	1.00	1.00	1.00	0.00	0.00	0.00	0.00	0.48	0.00	1.00	1.00	1.00	1.00	0.08	0.00	1.00	0.00	0.00	0.00	0.00
0.60	0.00	0.40	0.00	0.60	0.60	0.40	0.60	0.00	0.00	1.00	1.00	1.00	0.00	0.00	0.00	0.00	0.00	0.00	1.00	1.00	1.00	1.00	1.00	0.00	1.00	0.00	0.00	0.00	0.00
0.33	0.02	0.67	0.02	0.69	0.06	0.31	0.06	0.00	0.00	1.00	1.00	1.00	0.00	0.04	0.00	0.00	0.56	0.00	0.00	0.04	0.00	0.00	0.00	1.00	0.04	0.96	1.00	0.12	0.04
0.41	0.01	0.59	0.01	0.86	0.02	0.14	0.02	0.00	0.00	1.00	1.00	1.00	0.00	0.62	0.00	0.00	0.18	0.03	0.38	1.00	0.38	0.38	0.03	0.62	1.00	0.00	0.62	0.62	0.62
0.34	0.02	0.66	0.02	0.68	0.06	0.32	0.06	0.00	0.00	1.00	1.00	1.00	0.00	0.00	0.00	0.00	0.58	0.00	0.00	0.00	0.00	0.00	0.00	1.00	0.00	1.00	1.00	0.09	0.00
0.33	0.02	0.67	0.02	0.69	0.06	0.31	0.06	0.00	0.00	1.00	1.00	1.00	0.00	0.04	0.00	0.00	0.56	0.00	0.00	0.04	0.00	0.00	0.00	1.00	0.04	0.96	1.00	0.12	0.04
0.32	0.01	0.68	0.01	0.80	0.48	0.20	0.48	0.00	0.00	1.00	1.00	1.00	0.00	0.32	0.00	0.00	0.12	0.02	0.00	0.32	0.00	0.00	0.00	1.00	0.32	0.68	1.00	1.00	0.32
0.27	0.01	0.73	0.01	1.00	0.00	0.00	0.00	0.00	0.00	1.00	1.00	1.00	0.00	1.00	0.00	0.00	0.00	0.05	0.00	1.00	0.00	0.00	0.00	1.00	1.00	0.00	1.00	1.00	1.00

Fig. 3. A similarity matrix Λ

TABLE 1
Sorted mutants with killed difficulties

ID	M_i	$Dif(M_i)$	ID	M_i	$Dif(M_i)$
1	M_{14}	0.9998	2	M_{16}	0.9998
3	M_{17}	0.9998	4	M_{19}	0.9990
5	M_{24}	0.9990	6	M_{20}	0.9870
7	M_{22}	0.9870	8	M_{23}	0.9870
9	M_2	0.9850	10	M_4	0.9850
11	M_{30}	0.9792	12	M_{15}	0.9790
13	M_6	0.9688	14	M_8	0.9688
15	M_{21}	0.9662	16	M_{26}	0.9662
17	M_{29}	0.9356	18	M_{18}	0.7076
19	M_7	0.6818	20	M_1	0.6600
21	M_9	0.5784	22	M_{10}	0.5286
23	M_{27}	0.5054	24	M_{25}	0.4846
25	M_{28}	0.4846	26	M_{12}	0.4714
27	M_{13}	0.4714	28	M_{11}	0.4714
29	M_3	0.3404	30	M_5	0.3182

of C_k. Let $Th \in$ [0,1] be a threshold which is a constant. If $\alpha_{i,j} \geq Th$, we will put M_j in C_k, followed by deleting M_j from M^S. In this way, M_j can be arranged into the multiple clusters by fuzzy clustering. Such a process will be repeated until M^S becomes empty (indicated by the while loop in lines 2 to 17) and thus m clusters are generated. In $C_k = \{M_1^k, M_2^k, \cdots, \}$, where M_1^k is also the cluster center.

Note that the threshold, Th, in Algorithm 1 cannot be

Algorithm 1 Fuzzy clustering of sorted mutants

Input: $M = \{M_1, M_2, ..., M_n\}$ (a mutant set), $M^S = \{M_1^S, M_2^S, ..., M_n^S\}$ (a sorted set of the mutants), Λ (similarity matrix)
Output: $C_1, C_2, ..., C_m$ (m clusters)
1: Set k=1
2: **while** $M^S \neq \emptyset$ **do**
3: Set $C_k = \emptyset$
4: Select the first element M_1^S from M^S as center of C_k
5: $C_k = C_k \cup \{M_1^S\}$, where $M_i \in M$ and $M_i = M_1^S$
6: $M^S = M^S \backslash \{M_1^S\}$
7: $M = M \backslash \{M_1^S\}$
8: Decrement n by 1
9: **for** $j = 1$ to n **do**
10: Obtain $\alpha_{i,j}$ between M_i and M_j ($M_i, M_j \in M$ and $i \neq j$) from Λ;
11: **if** $\alpha_{i,j} \geq Th$ **then**
12: $C_k = C_k \cup \{M_j\}$
13: $M^S = M^S \backslash \{M_h^S\}$, where $M_h^S = M_j$
14: **end if**
15: **end for**
16: Increment k by 1
17: **end while**

too high or too small. If Th is too small, it implies that many mutants may be considered to be similar to the cluster center. Such a setting may cause a situation where different clusters have lots of overlaps; in other words, these clusters will not be so different, which is contradictory to the intuition of clustering. If Th is too high, fewer mutants will be removed from M^S in each round. As a result, there may be a large number of clusters, which will increase the computation overhead and thus deteriorate the efficiency of test data generation. We also conduct some preliminary experimental studies and confirmed, after many times of trial and error, we obtain an appropriate value of Th. In this paper, Th is set to 0.5.

To illustrate the fuzzy clustering of mutants, let us look at the 30 mutants in Fig. 1 again. Based on Table 1, we can select M_{14} (which is the most difficult to be killed) as the center for cluster C_1. Then, according to similarity matrix Λ in Fig. 3, we can select the non-center mutants M_3, M_5, M_{11}, M_{12}, M_{13}, M_{15}, M_{16} and M_{17} (whose similarities to M_{14} are higher than 0.5) into C_1. Since M_{16} and M_{17} (the second and third elements in the original M^S) have been selected into C_1 and thus removed from M^S, M_{19} will be selected as the center for constructing the next cluster C_2. Such a process is repeated, and finally we can obtain seven clusters, as given in Column 1 in Table 2.

3.4 Optimization model for test data generation

Let the decision vector of the optimization problem be the input vector of a program, i.e., X.

3.4.1 Objective function

Since our goal of generating test data is to kill a mutant, the objective function should reflect whether a test datum can kill a mutant. For mutant M_i^k (the i-th mutant in the k-th cluster), our goal is to find a test datum to kill it. Thus, its objective function is denoted as $f_i^k(X)$. If a decision vector, X, can kill M_i^k, $f_i^k(X) = 0$; otherwise, $f_i^k(X)$= 1. As a result, M_i^k is killed if and only if $f_i^k(X)$ takes the minimum value of zero. In this way, the problem of killing M_i^k can be transformed to the optimization problem of minimizing $f_i^k(X)$:

$$\min f_i^k(X)$$

However, the value of $f_i^k(X)$ is either 0 or 1. Such a function has difficulty in guiding the evolution of a population. To provide more information for the evolution of a population, we add a constraint to the model.

3.4.2 Constraint function

Previous studies have shown that test data must first satisfy the reachability condition before killing a mutant, that is, a test datum must first cover the mutated statement, s', before killing M_i^k. Thus, the constraint function of our optimization model is defined based on the branch coverage [27], which is expressed as:

$$g_i^k(X) = Appr(s', X) + (1 - 1.001^{-dist(s', X)}), \quad (7)$$

where $Appr(s', X)$ is the layer proximity of X for s', and $dist(s', X)$ means the branch distance. As can be seen from Eq. (7), X can cover s' if and only if $g_i^k(X) = 0$. As a result, we can take $g_i^k(X) = 0$ as the constraint of the model.

3.4.3 Optimization model

According to X, $f_i^k(X)$ and $g_i^k(X)$, we establish the optimization model of generating test data for killing M_i^k, expressed as:

$$\min f_i^k(X) \quad \text{s.t.} \begin{cases} g_i^k(X) = 0 \\ X \in D \end{cases}. \quad (8)$$

Eq. (8) suggests that there are n models corresponding to n mutants.

3.5 MGA for generating test data killing multiple mutants

Compared with other search-based methods, GAs are widely employed in generating test data [22], [26], [40], [41]. Generally speaking, SGA only generates a test datum targeting at one mutant when running once, which is inefficient for multiple clusters. MGA is an improved version of SGA for efficiently solving complex optimization problems.

Given that the mutants have been divided into a number of clusters, *MGA via individual sharing* [30] is a good solution in generating test data that kill multiple mutants across different clusters. Algorithm 2 gives the details for generating test data based on MGA.

In Algorithm 2, the number of sub-populations in *MGA* is the number of clusters, i.e., m. A sub-population optimizes a subproblem of generating test data for a center mutant in a cluster. All sub-populations evolve in parallel.

In line 1 of Algorithm 2, when initializing m sub-populations, MGA selects the initial individuals from the initial test data which have been used in calculating the killed difficulty and similarity in **Step 2**, which helps to improve the quality of the initial population..

For a given cluster, C_k, after the individuals in the k-th population executing the center mutant M_1^k (line 8), there will be three cases as follows:

1) There exists a test datum X_ℓ^k that can kill M_1^k as well as all other non-center mutants in the cluster.
2) There exists a test datum X_ℓ^k that can kill M_1^k, but X_ℓ^k can only kill part of non-center mutants in the cluster.
3) M_1^k cannot be killed by any individual.

For case 1, we stop the evolution of the corresponding sub-problems (line 12).

For cases 2 and 3, we delete the killed mutants in C_k, a new hard-killed mutant is select from C_k as the cluster center and run MGA again (line 29).

In addition, if all individuals in the k-th population cannot kill M_1^k, MGA needs to perform genetic operations (lines 21 to 24) before running the next generation (line 27). The genetic operations are roulette wheel selection, one-point crossover, and one-point mutation. The probabilities of crossover and mutation operations are 0.9 and 0.3, respectively.

TABLE 2
Information of clusters and alive mutants when Th =0.5

Clusters	Test data	Alive mutants
$C_1(M_{14}) = \{M_{14}, M_3, M_5, M_{11}, M_{12}, M_{13}, M_{15}, M_{16}, M_{17}\}$	(39,36,15)	M_3
$C_2(M_{19}) = \{M_{19}, M_3, M_5, M_{11}, M_{12}, M_{13}, M_{15}, M_{21}, M_{25}, M_{26}, M_{28}, M_{29}, M_{30}\}$	$(15, 14, 15)$	M_5
$C_3(M_{24}) = \{M_{24}, M_1, M_5, M_6, M_8, M_{11}, M_{12}, M_{13}, M_{20}, M_{21}, M_{22}, M_{23}, M_{26}\}$	(63,62,62)	M_1
$C_4(M_2) = \{M_2, M_1, M_4, M_5, M_{11}, M_{12}, M_{13}, M_{25}, M_{27}, M_{28}\}$	(8,8,7)	M_5,M_{27}
$C_5(M_{18}) = \{M_{18}, M_3, M_5, M_{11}, M_{12}, M_{13}, M_{25}, M_{27}, M_{28}\}$	(29,19,47)	M_3, M_5
$C_6(M_7) = \{M_7, M_3, M_{11}, M_{12}, M_{13}, M_{25}, M_{27}, M_{28}\}$	(20,10,38)	$M_{11}, M_{12}, M_{13}, M_{25}, M_{27}, M_{28}$
$C_7(M_9) = \{M_9, M_3, M_5, M_{10}\}$	(20,10,38)	M_5
Test suite T={(39,36,15),(15,14,15),(63,62,62),(8,8,7),(29,19,47),(20,10,38)}		

Algorithm 2 Generating test suite using MGA

Input: Pop (population including $m(m')$ sub-populations), $Size$ (sub-population size), g (maximum number of iterations)
Output: T (Test suite)

1: Initialize m sub-populations and the values of all parameters;
2: Set *count*= 1
3: **while** *count* $\leq g$ **do**
4: **while** $m(m')$ ≠0 **do**
5: Set k=1
6: **while** $k \leq m(m')$ **do**
7: **for** k =1 to $m(m')$ **do**
8: $X_1^k,\ X_2^k,\ \cdots,\ X_{Size}^k$ execute cluster center M_1^k
9: **if** X_ℓ^k can kill M_1^k **then**
10: X_ℓ^k will execute alive non-center mutants in C_k
11: **if** X_ℓ^k can kill all mutants **then**
12: Stop the evolution of the k-th sub-population
13: Decrement m by 1
14: **end if**
15: Save the killed mutants and X_ℓ^k
16: **end if**
17: **end for**
18: Increment k by 1
19: **end while**
20: **for** $k = 1$ to $m(m')$ **do**
21: **if** all individuals in the k-th sub-populations cannot kill M_1^k **then**
22: Calculate fitness $fit(X_\ell^k)$ of each individual
23: Perform selection, crossover, and mutation;
24: Generate new individuals
25: **end if**
26: **end for**
27: Increment *count* by 1
28: **end while**
29: Select a new center from each cluster
30: Obtain $m'(\leq m)$ sub-populations
31: **end while**

If there are still alive mutants after running MGA once, we should update the sub-populations with the changed $m'(\leq m)$ clusters (lines 30). The process above is repeated, until termination criteria are met.

Algorithm 2 adopts two termination criteria. One is that the desired test data are generated for m clusters, i.e, the number of optimization problems is 0 (line 4). The other termination criterion is that the population evolves for the maximum number of iterations, g (line 2).

Note that the individual sharing in MGA is realized through a loop structure (lines 7 to 19). In other words, we not only determine whether an individual ($X_1^k,\ X_2^k,\ \cdots,\ or\ X_{Size}^k$) is an optimal solution (a generated test data) of the corresponding the k-th cluster, but also judge whether these individuals can kill the mutants in the other clusters. In this way, the efficiency of finding optimal solutions for each mutant can be improved, whereas the complexity of the algorithm will not be increased significantly.

Moreover, the fitness of every individual is calculated, as shown in line 22. Given that our model includes one objective function, $f_i^k(X)$, and one constraint function, $g_i^k(X)$ in Eq.(8), we design a fitness function by the penalty function method, which is typical in tackling constrained optimization problems, expressed as:

$$fit_i^k(X) = f_i^k(X) \times (g_i^k(X) + \varphi), \tag{9}$$

where φ is a small positive constant that the value in parentheses is greater than 0. As a result, X can kill a mutant if and only if $fit_i^k(X) = 0$.

Note that the weak mutation conditions are only embedded in the unique constraint. In contrast, the objective function guarantees that the desired test data can kill the mutant. The fitness function can guide a population to generate test data using genetic algorithms. Surely, to requiring more information when evolving a population, we will design a fitness function by adding the propagation condition, which is generally more sufficient, and we will do that in the future.

Refer to the example given in Fig. 1 again. The decision vector is the input vector $X = (x, y, z)$. Initially, let the number of the sub-populations $m = 7$ which is the number of clusters, and the population size, $Size = 5$. We initialize each sub-population and set the largest generation, $g = 3000$.

If a test datum can kill M_{14} in C_1, the test datum must first cover s' (*if* $(x+x+y*y == z*z)$) in M_{14}. For example, in a certain generation, individual $X = (2, 3, 15)$ executes

M_{14}, we obtain $g(X) = 1 + (1 - 1.001^{-(15-(2+3))}) \neq 0$, based on Eq.(8), suggesting that $(2, 3, 15)$ cannot cover s'. Thus, $X = (2, 3, 15)$ also cannot kill M_{14}, i.e., $f(X) = 1$. if all the individuals cannot kill M_{14}, selection, crossover and mutation are performed.

After evolving some generations, $X = (39, 36, 15)$ makes $g(X) = 0$ and $f(X) = 0$, thus it is the test datum killing M_{14}. Next, we employ $(39, 36, 15)$ to execute the non-center mutants in C_1, and all non-mutants but M_3 are killed. Similarly, after executing MGA once, the test data and the alive mutants are generated for each cluster, which are listed in Table 2.

Note that although some mutants are alive in one cluster, they can be killed in another cluster. For example, M_3 cannot be killed in cluster C_1, but it can be killed by the test data for C_2, C_6 and C_7, suggesting that fuzzy clustering contributes to killing non-center mutants. The final test suite killing all thirty mutants are obtained, listed on the last row of Table 2.

4 Experimental Study

In this section, we confirm the performance of **FUZGENMUT**, by applying it to nine benchmark and industrial programs under test. The research questions in the experiments are first presented. Then, the experimental setups are briefly illustrated. Following that, we introduce the experimental environment and process. Finally, the experimental results are provided and analyzed.

4.1 Research Questions

To illustrate the effectiveness of **FUZGENMUT**, we raise the following three questions.

RQ1: Does fuzzy clustering help bolstering the effectiveness of mutation testing?
It is not surprising that one mutant may be similar to other mutants from different clusters. Therefore, it is natural to consider fuzzy clustering when we divide the mutant set into different groups. In this study, we examine whether and to what extent the overlapping among different clusters (the expected result of fuzzy clustering in **FUZGENMUT**) affects the efficacy of mutation testing.

RQ2: Does *MGA* in **FUZGENMUT** increase the efficiency of test data generation for mutation testing?
A key intelligent technique used in **FUZGENMUT** is the MGA via individual sharing. Although it has shown a high potential in various fields, its applicability and effectiveness in mutation testing are yet to be assessed, which would be done in this study.

RQ3: To what extent does the combination of sorting and clustering improve the performance of mutation testing?
In **FUZGENMUT**, fuzzy clustering is implemented based on the sorted set of mutants, where the most "hard-to-kill" mutants are selected as the cluster center. Intuitively speaking, such a sorting plus clustering scheme should deliver a high performance. A series of experiments are conducted to examine the validity of the intuition.

4.2 Experimental Setup

Our experiments are conducted on a machine with 2 $\times$ Intel(R) Core(TM) i5 CPU and 4GB memory. The operating system is Microsoft Windows 7.

4.2.1 Object programs

We select nine object programs across a wide range of application domains, data types, logic structures, functionalities, and scales. The basic information of these programs is given in Columns 1–3 and 7 in Table 3. Among them, $G1$ – $G4$ are relatively small programs [33], [44], [45], [46], which implement some simple calculations. $G5$ and $G6$, two medium-sized programs, are from the famous Siemens suite, which have been widely used by different researchers [31], [33], [45]. $G7$ – $G9$ are large-sized programs. Among them, $G7$ is the widely studied European Space Agency program, while $G8$ and $G9$ are the well-known UNIX utilities [31], [33], [44].

All the selected object programs are written in C language, and can be downloaded from the Software Infrastructure Repository (http://sir.unl.edu/portal/index.php).

4.2.2 Mutant Generation

The basic information of the mutants for each object program is given in Columns 4–6 in Table 3. We generate the mutants through the following three major steps:

First, we select the statements to be mutated. In our experiments, we deliberately selected part of statements for each object program, by considering various factors, such as the structure complexities, types of statements, and mutation operators [47]. As summarised in Column 4 in Table 3, around 30% of the statements are selected to generate mutants for each object program.

The second step is seeding faults by mutation operators on the selected statements. We select 13 classes of mutation operators [32], as summarized in Table 4 in the experiments. Yao et al. [33] clarified that while almost a half (47%) of mutants generated using the mutation operator, ABS, are equivalent. In addition, the mutants of LCR operator are few equivalent mutants (only 2%). Therefore, we generate a smaller number of ABS mutants and a larger number of LCR mutants in the experiments.

The final step is to identify the equivalent mutants. There are various tools for generating mutants [4], [34], [36], [37]. However, a key open question is how effective these tools are and how reliable the studies are based on them [4]. Thus, we manually identifying equivalent mutants by many individuals. This step is actually interleaved with the above second step of applying mutation operators. Once a mutation operator is applied to a selected statement, we would judge whether the resultant mutant is syntactically equivalent to the program under testing. In the experiments, if an individual was not sure about whether a mutant is equivalent, he/she would ask for others help to avoid a human judgment bias.

Column 6 gives the number of non-equivalent mutants for each object program. In addition, as a part of Step 2, we calculate the killed difficulty of each non-mutants by the initial test suite. The average killed difficulty for each program is given in Column 5 of Table 5, suggesting that these generated mutants is more difficult to kill.

TABLE 3
Information about the object programs

ID	Programs	lines of code	# of statements under test	# of mutants	# of non-equivalent mutants	Function
$G1$	Profit	24	8	56	45	Commission of the salesperson
$G2$	Insert	35	10	42	29	Insert sort
$G3$	Day	42	13	65	54	Calculating the order of the day
$G4$	Calendar	137	45	155	129	Calendar calculation
$G5$	Totinfo	406	135	607	486	Information statistics
$G6$	Replace	564	188	912	757	Pattern matching
$G7$	Space	9564	3188	9658	7243	Array language interpreter
$G8$	Flex	10459	3480	11924	9778	Unix lexer utility
$G9$	Make	35545	9664	14952	11812	Unix compilation utility
Sum	-	56776	16731	38371	30333	-

TABLE 4
Information of mutation operators

ID	Mutation Operator	Description
1	ABS	absolute value insertion
2	AOR	arithmetic operator replacement
3	CAR	constant for array reference replacement
4	CRP	constant replacement
5	CSR	constant for scalar variable replacement
6	LCR	logical connector replacement
7	ROR	relational operator replacement
8	RSR	RETURN statement replacement
9	SCR	scalar for constant replacement
10	SAR	scalar variable for array reference replacement
11	SRC	source constant replacement
12	SVR	scalar variable replacement
13	UOI	unary operator insertion

TABLE 5
Information of initial test suite

ID	R	Alive mutants	MS_{weak}	Ave. of $Dif(M_i)$
$G1$	500	1	97.78%	74.23
$G2$	500	0	100%	68.45
$G3$	500	3	96.30%	67.15
$G4$	500	5	96.12%	71.43
$G5$	1500	20	95.88%	78.56
$G6$	1500	32	95.77%	76.34
$G7$	5000	223	96.92%	79.75
$G8$	5000	392	95.98%	81.78
$G9$	5000	562	95.24%	89.11
$Ave.$	–	–	96.67%	76.31

4.2.3 Generation of initial test suite

In MGA, the initial test data,which is generated to calculate the killed difficulty and similarity in , are employed as the individuals of the initial population. The basic information of the initial test suites is summarized in Table 5. For each program under test, we set a constant, R (in Eq.(3)), as the size of initial test suite, where $R = 500, 1500$ and 5000 for small-, medium-, and large-scale programs, ($G1$ – $G4$, $G5$ – $G6$, and $G7$ – $G9$), respectively, as given in Column 2 of Table 5. R is set according to the information of a program and the number of mutants by experiences as well as previous studies [1,5,44,48]. Although there exist some mutants that cannot be killed by the initial test suite ("Alive mutants" in Column 3 of Table 5), the mutation score under the weak mutation criterion denoted by MS_{weak}, is already very high (more than 95%) for each program (Column 4 of Table 5), suggesting that the initial test suite can meet the requirements of calculating the killed difficulties and the similarities although it may have no good performance in diversity and coverage. MS_{weak} is selected as a metric to reflect the sufficiency of the initial test suite generated by the random method, which is equal to the killing rate of mutants by the initial test data. The threshold of MS_{weak} is set to 95. In the experiments, the initial test data are successively generated by the random method and execute a program with mutant branches, then the value of MS_{weak} is calculated. Such a process is repeated until $MS_{weak} \geq 95$. If $MS_{weak} < 95$ within the specified size of the test suite, we will manually generate some test data.

4.3 Experimental Process

4.3.1 Procedure for RQ1

Fuzzy clustering (denoted by **FC**) is one of the two key technologies in **FUZGENMUT**. To evaluate its effectiveness in the experiment, we select the *hard clustering* (**HC**) as the benchmark for comparison. In **HC**, once a mutant is selected into a cluster, it will be removed from both M (the mutant set) and M^S (the sorted set of mutants) to guarantee that one mutant can belong to one and only one cluster.

Note that since RQ1 is focused on the effectiveness of the clustering techniques, the MGA algorithm is run only once to generate test data after the clustering process in **HC**. To have a fair comparison, we also include the **FUZGENMUT** with one run of MGA, simply denoted by **FC**, in the experiments for RQ1.

In the experiment, the sub-population size of MGA in **FUZGENMUT** is $size = 5$, the maximum number of iterations is $g = 3000$. The genetic operations are roulette wheel selection, one-point crossover, and one-point mutation. The probabilities of crossover and mutation operations are 0.9 and 0.3, respectively. The other parameter settings and termination criteria can be referred to Section 3.5.

Two metrics are used for answering RQ1. One is the mutation score, MS, defined as Eq. (1), which evaluates the fault-detection effectiveness of test data generated by **FC**, **HC** and **FUZGENMUT**. The other metric is called the *killing rate on non-center mutants*, denoted as KR_c, which refers to the ratio of the number of killed non-center mutants to the total number of non-center mutants, that is,

$$KR_c = \frac{\text{No. of killed non-center mutants}}{\text{No. of all non-center mutants}}. \quad (10)$$

For a fair comparison, we employe KR_c to compare **HC**

and **FC** only, that is, the two techniques with one run of MGA.

Intuitively speaking, the higher value MS or KR_c has, the higher the effectiveness of the corresponding method is.

4.3.2 Procedure for RQ2

The second key technology of **FUZGENMUT** is to employ MGA to generate test data in Step 6 after sorting and clustering the mutants. To evaluate the efficiency of MGA, we select two baseline techniques. One is the random method (**RD**) for generating test data. The number of test data generated by **RD** is set 3000, the same as the maximum number of iterations in **FUZGENMUT**. The other benchmark is **SGA** (single population genetic algorithm). Since **SGA** generates test data from one single population, it does not require the clustering or sorting of mutants. **SGA** is also a traditional search-based technique of generating test data for mutation testing. In our experiments, except the number of sub-populations (which is one), its parameter settings and the termination criteria are the same as those of MGA in **FUZGENMUT** (given in Section 3.5).

In each generation, MGA in **FUZGENMUT** can generate test data for killing multiple mutants in parallel. By contrast, based on **RD** and **SGA**, test data are successively generated for killing mutants one by one.

To compare the performance of **RD**, **SGA** and MGA in **FUZGENMUT**, in addition to MS, we employ two other metrics, *time consumption in test data generation* and *the number of iterations*. Intuitively speaking, if a method requires a shorter time for generating all test data, the method is considered to be better. The fewer iterations are, the more efficient the corresponding method is.

4.3.3 Procedure for RQ3

The combination of sorting and clustering in Steps 3 and 4 is also an important part of **FUZGENMUT**. To answer RQ3, we examine and compare the performance of four techniques based on different combinations.

SC (**sorting and clustering**): The combination of sorting and clustering is employed on the mutants.

S¬C (**sorting only**): The mutants are not clustered. Test data are always generated to kill the first mutant in M^S, and then we will judge whether the generated test data can kill other alive mutants. All killed mutants will be removed from M^S. Such a process is repeated until the termination criteria are satisfied.

¬SC (**clustering only**): The mutants are not sorted. A mutant will be randomly selected as a cluster's center (instead of the first element in M^S, which does not exist in **¬SC**). Then, we generate test data to kill the center mutant, and judge whether it can kill other mutants in the same cluster.

¬S¬C (**neither sorting nor clustering**): The method is a naive implementation of mutation testing. A mutant is randomly selected one by one from the mutant set. After generating a test datum to kill it, we will judge whether it can kill other mutants.

To have a fair comparison, all four techniques used **SGA** to generate test data instead of MGA in **FUZGENMUT**, because MGA cannot be used in **S¬C** or **¬S¬C** (no clustering), which do not be provided multiple sub-populations.

In addition to MS and the time consumption in test data generation, we also measured the *number of test data* generated by each of these four techniques. Given the similar MS, a method can be regarded as more efficient if it can generate a smaller number of test data within a very short time.

To further investigate the influence of sorting on the clustering performance, we compared **SC** and **S¬C** based on three metrics, namely the *cluster compactness* (CP), the *cluster separation* (SP) and the *clustering rate* (CR).

CP refers to the average distance between the center data and the non-center data (mutants) in the same cluster. In this paper, the similarity between the two mutants is associated with their distance. Apparently, the smaller the similarity between the two mutants is, the larger their distance is. For cluster C_k, its compactness is calculated as

$$CP_k = \frac{1}{c_k}\sum_{j=2}^{c_k}(1-\alpha_{i,j}), \tag{11}$$

where c_k is the number of mutants in C_k, and $\alpha_{i,j}$ is the similarity between M_i and M_j as defined in Section 3.1. For all clusters, the compactness is calculated as

$$CP = \frac{1}{m}\sum_{k=1}^{m} CP_k, \tag{12}$$

where m is the number of clusters. Intuitively speaking, the smaller the value of CP is, the closer the mutants in the same cluster are to one another.

SP is defined as the average distance between each pair of cluster centers, defined as follows:

$$SP = \frac{2}{m^2 - m}\sum_{r=1}^{m}\sum_{o=r+1}^{m}(1-\alpha_{r,o}) \tag{13}$$

In general, the larger the value of SP is, the farther the clusters are away from one another.

CR refers to the ratio of the number of clusters to the total number of mutants, that is,

$$CR = \frac{\text{No. of clusters}}{\text{No. of all non-equivalent mutants}} \tag{14}$$

Apparently, the smaller the value of CR is, the fewer clusters there will be, and thus the lower the corresponding computation overhead will be.

5 Experimental Results

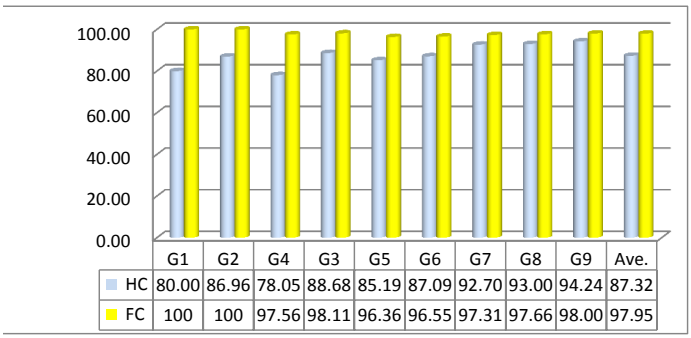
5.1 Answer to RQ1

Fig. 4 shows the results of comparing KR_c and MS of **HC**, **FC** and **FUZGENMUT**.

In Fig. 4(a), we only include the values of KR_c of **FC** and **HC**, which are collected after one run of MGA. It can be observed that the average value of KR_c of **FC** (97.95%) is 10% higher than that of **HC** (87.32%). This observation indicates that **FC** can help to kill more non-center mutants.

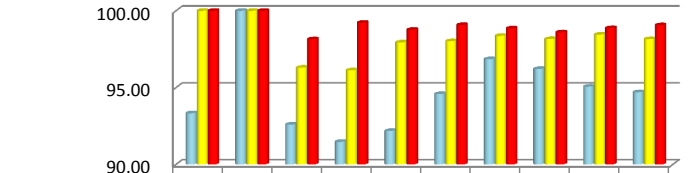
The results of MS given in Fig. 4 (b) show that the average value of MS of **FC** (98.15%) is 4% greater than that of **HC**(94.70%), suggesting that **FC** is more capable in killing the mutants.

To verify the significant superiority of **FC**, we employe the Mann-Whitney U test. Let the significance level of the U



	G1	G2	G4	G3	G5	G6	G7	G8	G9	Ave.
HC	80.00	86.96	78.05	88.68	85.19	87.09	92.70	93.00	94.24	87.32
FC	100	100	97.56	98.11	96.36	96.55	97.31	97.66	98.00	97.95

(a) The values of KRc based on FC or HC by running MGA once



	G1	G2	G3	G4	G5	G6	G7	G8	G9	Ave.
HC	93.33	100	92.59	91.47	92.18	94.58	96.85	96.22	95.06	94.70
FC	100	100	96.30	96.12	97.94	98.02	98.37	98.16	98.45	98.15
FUZGENMUT	100	100	98.15	99.22	98.77	99.08	98.85	98.60	98.87	99.06

(b) The values of mutation score

Fig. 4. Performance of **FC**, **HC** and **FUZGENMUT**

test be 0.05. The results of the Mann-Whitney U test show that **FC** is significantly superior to **HC** in terms of KR_c and MS.

Fig. 4(b) also shows that MS of **FUZGENMUT** is only marginally higher than that of **FC**. It implies that even with only one run of MGA, **FC** still can deliver very high fault-detection effectiveness.

In summary, we can answer RQ1 that fuzzy clustering delivers high effectiveness of mutation testing.

5.2 Answer to RQ2

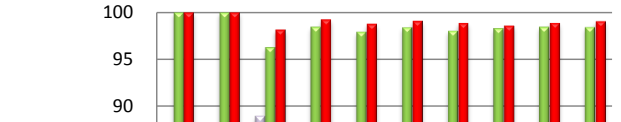
When generating test data using MGA in **FUZGENMUT**, **SGA** and **RD**, we independently run each algorithm 30 times for each mutant in the experiments, and take the average of these results when evaluating their performance. The comparison results among **FUZGENMUT**, **SGA** and **RD** are given in Figs. 5 to 7.

Fig. 5 shows the MS values of three techniques. On average, the MS value of **FUZGENMUT** is 15.55% higher than that of **RD**, and 0.63% more than that of **SGA**. Take $G8$ for an example, **FUZGENMUT** kills 1769 more mutants than **RD**, and 166 more than **SGA**. Based on the data of **FUZGENMUT** and **RD** in Fig. 5, the values of *Mann-Whitney U* show that **FUZGENMUT** is significantly superior to **RD** in terms of MS.

Fig. 6 shows that the time consumption in generating test data by three technologies. Note that Fig. 6 shows the results of $G1$ – $G4$, $G5$ & $G6$ and $G7$ – $G9$ respectively, due to the large discrepancy in the scale. It can be observed that **RD** spends the shortest time in generating test data for all programs with the lowest values of MS (refer to Fig. 5). These observations suggest that although it could be quick, the randomly generated test data are insufficient for mutation testing. Therefore, we focus on comparing **FUZGENMUT** and **SGA**. As can be observed from Fig. 6, **FUZGENMUT** spends much shorter time than **SGA** in generating test data. Based on the data of **FUZGENMUT** and **SGA** in Fig. 6.

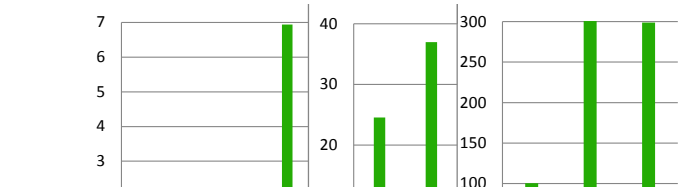
Fig. 7 demonstrates the average number of evaluations required in test data generation for **FUZGENMUT** and **SGA**. For all programs, **FUZGENMUT** requires fewer number of evaluations than **SGA**, though the results of Mann-Whitney U test do not show the significance of their difference.

In summary, **FUZGENMUT** outperform **SGA**, not only due to its higher MS, but also because of its shorter time and fewer number of evaluations in test data generation. On the other hand, although **FUZGENMUT** has marginally longer time consumption than **RD**, its MS is much higher, implying a better overall performance. The reason why **FUZGENMUT** is superior is that MGA in **FUZGENMUT** can generate test data that kill more than mutant in different clusters in parallel at each iteration, whereas **RD** and **SGA** can generate only one test datum targeting at one mutant selected from the mutant set. In this way, they successively generate test data that kill the disordered mutants.



	G1	G2	G3	G4	G5	G6	G7	G8	G9	Ave.
RD	84.44	82.76	88.89	83.72	82.10	82.34	81.58	81.91	83.81	83.51
SGA	100	100	96.30	98.45	97.94	98.41	98.04	98.30	98.46	98.43
FUZGENMUT	100	100	98.15	99.22	98.77	99.08	98.85	98.60	98.87	99.06

Fig. 5. MS (%) of **FUZGENMUT**, **SGA** and **RD**

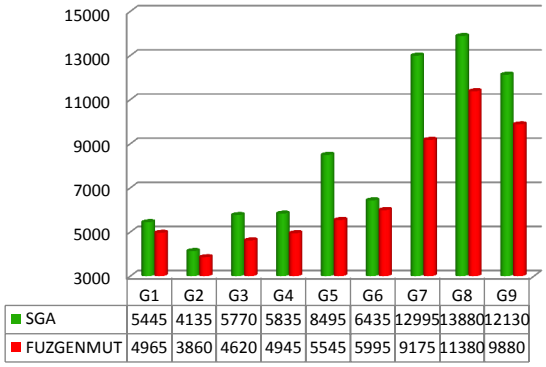


	G1	G2	G3	G4	G5	G6	G7	G8	G9
RD	0.09	0.03	0.07	0.49	1.42	3.89	19.27	25.89	24.52
SGA	0.88	0.45	1.19	6.94	24.55	36.96	100.45	300.78	298.77
FUZGENMUT	0.17	0.09	0.18	1.16	3.40	6.10	37.38	52.44	49.11

Fig. 6. Time consumption in test data generation (in second) of **FUZGENMUT**, **SGA** and **RD**

5.3 Answer to RQ3

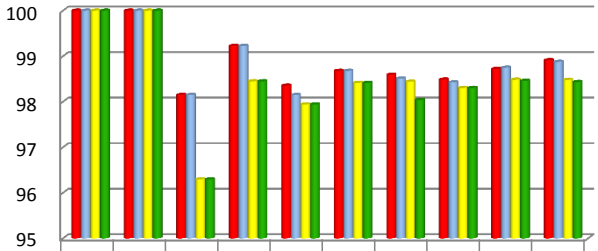
Figs. 8 to 10 demonstrate MS, time consumption in test data generation, and the number of test data of **SC**, **S**$^{\neg}$**C**, $^{\neg}$**SC**



	G1	G2	G3	G4	G5	G6	G7	G8	G9
SGA	5445	4135	5770	5835	8495	6435	12995	13880	12130
FUZGENMUT	4965	3860	4620	4945	5545	5995	9175	11380	9880

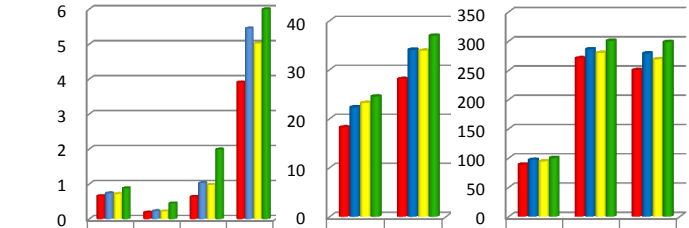
Fig. 7. Average number of evaluations for generating test data by **FUZGENMUT** and **SGA**

and $\neg\mathbf{S}\neg\mathbf{C}$, respectively.



	G1	G2	G3	G4	G5	G6	G7	G8	G9	Ave.
SC	100	100	98.15	99.22	98.35	98.68	98.59	98.49	98.72	98.91
S ¬ C	100	100	98.15	99.22	98.15	98.68	98.51	98.43	98.75	98.88
¬ SC	100	100	96.30	98.45	97.94	98.41	98.44	98.30	98.48	98.48
¬ S ¬ C	100	100	96.30	98.45	97.94	98.41	98.04	98.30	98.46	98.43

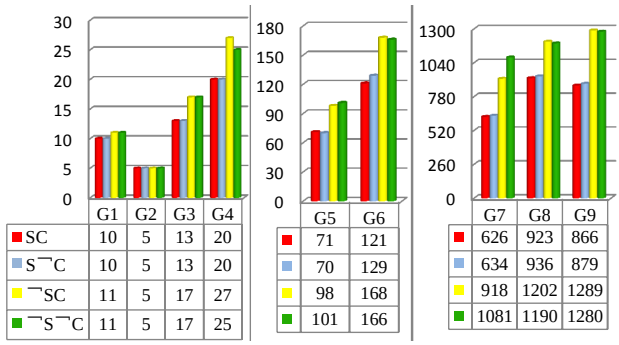
Fig. 8. MS for **SC**, $\mathbf{S}\neg\mathbf{C}$, $\neg\mathbf{SC}$ and $\neg\mathbf{S}\neg\mathbf{C}$



	G1	G2	G3	G4	G5	G6	G7	G8	G9
SC	0.66	0.19	0.64	3.91	18.23	28.10	89.03	270.77	250.61
S ¬ C	0.74	0.23	1.03	5.45	22.34	34.09	97.23	286.23	279.11
¬ SC	0.72	0.22	0.98	5.02	23.23	33.89	94.55	279.99	269.04
¬ S ¬ C	0.88	0.45	1.99	6.94	24.55	36.96	100.45	300.78	298.77

Fig. 9. Time consumption in test data generation (in second) for **SC**, $\mathbf{S}\neg\mathbf{C}$, $\neg\mathbf{SC}$ and $\neg\mathbf{S}\neg\mathbf{C}$

The values of MS in Fig. 8 clearly show that **SC** always has the highest fault-detection capability for all programs under test. However, the Mann-Whitney U test results do not show a significant difference among these four techniques in terms of MS. This is because all techniques apply the same fundamental strategy, SGA, to generate test data, while the order in the generation is different.



	G1	G2	G3	G4	G5	G6	G7	G8	G9
SC	10	5	13	20	71	121	626	923	866
S ¬ C	10	5	13	20	70	129	634	936	879
¬ SC	11	5	17	27	98	168	918	1202	1289
¬ S ¬ C	11	5	17	25	101	166	1081	1190	1280

Fig. 10. The number of test data for **SC**, $\mathbf{S}\neg\mathbf{C}$, $\neg\mathbf{SC}$ and $\neg\mathbf{S}\neg\mathbf{C}$

As observed in Fig. 9, **SC** always has the shortest time consumption in test data generation, not in the same order of magnitude as the other three techniques. $\neg\mathbf{S}\neg\mathbf{C}$ spends the longest time to generate test data. These observations indicate that the combination of sorting and clustering helps saving a lot of time for generating test data, especially for programs with a large number of mutants.

From Fig. 10, we can observe that **SC** always generates the smallest number of test data, which obviously much less than that of $\neg\mathbf{SC}$ and $\neg\mathbf{S}\neg\mathbf{C}$. This is because sorting mutants by their killed difficulty helps to generate high-quality test data. The test data that can kill those "hard-to-kill" mutants normally also have higher probabilities to kill other relatively "easy-to-kill" mutants.

Note that the computation overhead for executing mutation testing is normally in the order of the product of the number of mutants and the number of test data (naively speaking, executing each test datum on each mutant). Therefore, the smallest number of test data of **SC** naturally implies the lowest computation overhead for the execution of mutation testing. In other words, the combination of sorting and clustering in the proposed method helps to improve the efficiency of mutation testing.

To further investigate the impact of sorting on the clustering performance, we collect the values of CP, SP, and CR obtained by **SC** and $\neg\mathbf{SC}$, summarized in Table 6.

TABLE 6
CP, SP and CR of **SC** and $\neg\mathbf{SC}$

ID	CP		SP		CR(%)	
	SC	$\neg\mathbf{SC}$	**SC**	$\neg\mathbf{SC}$	**SC**	$\neg\mathbf{SC}$
$G1$	0.135	0.153	0.909	0.925	20.00	24.44
$G2$	0.177	0.153	0.931	0.953	20.69	20.69
$G3$	0.152	0.169	0.963	0.959	20.37	25.93
$G4$	0.102	0.133	0.965	0.972	16.28	20.93
$G5$	0.153	0.130	0.952	0.921	13.17	17.90
$G6$	0.160	0.179	0.959	0.922	14.53	19.95
$G7$	0.136	0.175	0.934	0.926	8.30	15.21
$G8$	0.147	0.176	0.946	0.902	9.17	16.33
$G9$	0.139	0.189	0.967	0.912	7.43	14.65
Ave.	0.146	0.163	0.952	0.933	13.74	18.95

As observed from Table 6, **SC** has better compactness, indicated by smaller CP, than $\neg\mathbf{SC}$ in seven out of nine cases. With respect to separation, **SC** outperformed $\neg\mathbf{SC}$ in six out of nine cases (larger SP). On average, **SC** is marginally superior to $\neg\mathbf{SC}$ in terms of CP and SP.

The difference between $\mathbf{SC}$ and $\neg\mathbf{SC}$ is very significant in terms of CR. On average, CR of $\mathbf{SC}$ is 1.38 (18.95/13.74) times smaller than that of $\neg\mathbf{SC}$.

Overall, we can answer RQ3 that the combination of sorting and clustering helps to improve the cost-effectiveness of mutation testing, and obtain a higher mutation score.

6 Threats to validity

The first threat to validity of this study is that the initial test suite might not be sufficient for precisely calculating the killed difficulty and the similarity among mutants. Thanks to the low cost of random testing and weak mutation, we can generate the initial test suite as large as possible such that the values of killed difficulty and mutant similarity could be as statistically reliable as possible.

The second threat comes from the uncertainty of clustering mutants. In the proposed method, if the similarity between two mutants is greater than a threshold (Th), they will be arranged to the same cluster. Therefore, the cluster of a mutant depends mainly on Th. We believe that the current setting ($Th = 0.5$) is quite fair, which is supported by our experimental studies (due to space limitations, we did not include the details of these secondary studies in the paper). However, it is still an outstanding question on how to precisely set an appropriate value of the threshold.

The third threat is related to the way of generating mutants. We applied a systematic and semi-automatic method to generate mutants to reduce the numbers of "easy-to-kill", redundant and equivalent mutants. Note that the manual work is unavoidable for experimental studies, especially when some statements and/or mutation operators are not suitable for mutating specific types of programs. To minimize the human bias in the experiments, we have taken all possible scenarios into consideration, and have involved with different individuals (not only the co-authors, but also other developers) to cross-check.

Last but not the least, the parameter settings of an algorithm have a threat to its efficiency, e.g., the population size of MGA in $\mathbf{FUZGENMUT}$. At present, there are no effective methods to set appropriate values of these parameters. We attempted to obtain reasonable values based on previous studies and experiences. How to correctly set these parameters for MGA is an important research topic for evolutionary computations, which is out of the scope of this paper.

7 Related work

The two key techniques in $\mathbf{FUZGENMUT}$ are the fuzzy clustering of mutants and the *MGA*-based test data generation. In this section, we discuss the related studies in two corresponding aspects, namely the mutant clustering and test data generation for mutation testing.

7.1 Mutant clustering

The number of mutants significantly affects the execution cost of mutation testing, because each mutant is supposed to be executed against at least one test data. Many researchers have attempted to reduce the number of mutants without a significant loss of testing effectiveness [47], [48], [49], [50].

Mutant sampling refers to that a small percentage of mutants is selected from the whole mutant set according to some criteria, sometimes even in a random manner. Jimenez et al. [8] analyzed the essence of a mutant, based on which a number of mutants are deliberately selected. However, the mutation score obtained from mutant sampling is normally reduced as the sampling rate decreases. In selective mutation [51], [52], only part of mutation operators are used when generating mutants, but its effectiveness is relatively low as compared with the technique using more mutation operators [11]. The high-order mutation can effectively reduce the number of mutants and simulate complex faults by combining a number of simple faults [53]. However, high computational complexity is often required for the generation of high-order mutants.

Different from the other three families of mutant reduction techniques, mutation clustering is a combination of data clustering and mutation analysis, to dramatically reducing both the numbers of mutants and test data [1], [54]. Recently, cluster analysis has become the main tool for exploratory data analysis and data mining. A major reason is that clustering algorithms and software are flexible in the sense that different mathematical frameworks are employed in the algorithms and a user can select a suitable method according to the application domain [16], [19]. For mutation testing, Hussain [16] proposed the idea of mutation clustering for the first time. He divided a mutant set into a number of subsets, i.e., clusters, based on the similarity of mutants. However, this method has some difficulty in determining the number of clusters and the initial cluster centers. Furthermore, this method generally has a low clustering accuracy, mainly due to the quality of clustering. To overcome this drawback, Huang [17] employed a *GA* to optimize the number of clusters and the initial cluster centers.

The methods of Hussain and Huang [16], [17] have a common characteristic that they define the similarity of mutants based on the Hamming distance between test data that kill the mutants. For some programs under test, some test data with small Hamming distance cannot kill similar mutants, which results in the inaccurate clustering of mutants. To address this issue, we give another definition of the similarity between the two mutants, which is based on the number of common test data that kill them. Also, we dynamically select cluster centers when clustering sorted mutants.

When clustering data, the relationship between the data may involve fuzzy boundaries, and thus the fuzzy clustering should be employed [28], [29], [55]. Fuzzy clustering has been widely studied and applied for different problems. Miyamoto et al. [28] proposed a clustering method based on fuzzy c-means, and compared crisp and fuzzy c-means as well as related techniques. Gath et al. [30] carried out fuzzy classification without a priori assumptions on the number of clusters in the data set, which was derived from a combination of the fuzzy K-means algorithm and the fuzzy maximum likelihood estimation. In this study, based on the observation that a mutant may be similar to mutants in more than one cluster. Under such a circumstance, it may cause some problems if a mutant belongs to only one cluster based on the hard clustering [16], [17]. Thus, we classify mutants

using fuzzy clustering, which helps to increase the killing probability of mutants.

In addition to mutant reduction, there exist other methods to reduce the cost of mutation testing. Previous studies have demonstrated that sorting mutants is an effective way to reduce the execution cost [14], [15]. Different from many other methods of sorting mutants, we obtain the priority of a mutant based on the killed difficulty based on the rationale that the test data capable of killing the hard-to-kill mutants should have a high quality. It has been shown that test data generated based on strong mutation generally have a high fault-detection effectiveness, whereas weak mutation is superior in reducing cost

7.2 Test data generation for mutation testing

Papadakis et al. [1] summarized three main approaches of test data generation for mutation testing, namely constraint-based test generation [56], dynamic symbolic execution [57], and search-based test generation [58], [59]. Among the studies related to the search-based approach, some work has been conducted to generate test data based on *GAs* under the criterion of mutation testing.

Masud et al. [41] presented a model of mutation testing based on *GAs*, which has good performance in detecting faults. Zhang [20] proposed the set evolution based on *GAs* to automatically generate test data by mutation analysis. The above studies suggested that *GAs* are competent in generating test data in the context of mutation testing.

However, the formulated optimization problem of generating test data that kill the mutants is normally large-scale, which increases as the number of mutants becomes higher. Under this circumstance, generating test data that kill mutants using a traditional *SGA* with only one population usually is very inefficient, due to at most one mutant being killed in each run of *SGA*. To improve efficiency, we generate test data for killing multiple mutants via *MGA*.

Another major challenge confronted when generating test data using the evolutionary algorithms is to design an appropriate model and a fitness function [27], [31], [41]. Kintis et al. [34] attempted to refine an approximate formula of the mutation score by mutant classification. Zhang et al. [14] predicted mutants to be killed by classifying them according to their features, which is helpful to accurately predict mutation execution results. Furthermore, Ayari et al. [59] designed a fitness function based on the reachability of a mutant, and Papadakis et al. [60] did the design in terms of the infection condition of a mutant. In **FUZGENMUT**, for the mutants after clustering, we formulate a new problem of generating test data for mutation testing as an optimization problem with a constraint, and generate test data to kill mutants using *MGA via individual sharing*.

8 Conclusion

Mutation testing has been used for evaluating the fault-detection effectiveness of testing methods [1], [5], [49]. Due to the involvement of a number of mutants, the implementation of mutation testing normally incurs a high cost, which, in turn, hinders its applicability and practicality. In this paper, we attempted to reduce the cost of test data generation for mutation testing by leveraging two intelligent technologies, fuzzy clustering and MGA. A comprehensive framework, namely **FUZGENMUT**, is thus proposed to generate test data in mutation testing with low execution cost. In **FUZGENMUT**, all mutants are first sorted according to their killed difficulties. The sorted mutants are then clustered. In the way, the test data killing the center mutant has a high probability of killing other mutants in the same cluster. In addition, since one mutant can belong to multiple clusters (that is, the so-called fuzzy clustering), The MGA is applied to support the simultaneous test data generation for different clusters, which further improves the efficiency of **FUZGENMUT**.

We evaluated the performance of **FUZGENMUT** on nine programs from different application domains and with various scales. The experimental results show that the fuzzy clustering helps to achieve a higher fault-detection capability of test data by comparing it with the traditional hard clustering. In addition, the sorting and clustering, working together, can significantly reduce the execution time for test data generation and decrease the number of the generated test data, while preserving a high mutation score. The MGA delivers much better performance than the baseline random method in killing mutants, while their execution time is comparable to each other. MGA also outperformed SGA in terms of both time consumption and the number of iterations when generating test data. All these results imply that intelligent technologies can help **FUZGENMUT** significantly enhance both the effectiveness and efficiency of mutation testing.

This study inspires quite a few research directions for future work. The proposed method (**FUZGENMUT**) consists of three parts, i.e., generating mutants, clustering mutants, and generating test data. Among them, the latter two parts, which are the core components of this paper, are realized automatically. The first part is not specific to this paper, but is generally suitable for all studies associated with mutation testing. As discussed in the response to the first and second comments, manual work is unavoidable for generating appropriate mutants, and we have alleviated this in the experiments. The issue related to manual work will be a part of our future studies to improve the automation of **FUZGENMUT**. **FUZGENMUT** can be considered as a general framework, each step of which can be improved by applying other advanced methods. For example, when calculating the killed difficulty and similarity, we can use the method of static analysis [35] or explore the path information of the program based on the module-depth and loop-depth rules [6]. In this way, we do not need to execute the weak mutation, which will further reduce the cost. What is more, the calculation results will not be affected by the initial test suite, which may help to remove the potential bias brought by concrete test data. In addition, this paper focuses on efficiently generating test data by employing fuzzy clustering and MGA, rather than comparing the performance of various metaheuristics. In the future, we will employ other advanced metaheuristics to generate test data. Moreover, studies can be conducted to see how to apply other intelligent technologies, e.g. different clustering algorithms and search-based strategies, into mutation testing, so as to further improve the performance of mutation testing.

ACKNOWLEDGMENT

This paper is jointly supported in part by National Key Research and Development Program of China (2018YFB1003802-01), and National Natural Science Foundation of China (61773384, 61573362 and 61503220). ∗ Corresponding author: D. Gong; X. Yao.

REFERENCES

[1] M. Papadakis, M. Kintis, and Z. J, "Mutation testing advances: an analysis and survey," *Advances in Computers*, vol. 112, pp. 275–378, 2019.

[2] R. G. Hamlet, "Testing programs with the aid of a compiler," *IEEE transactions on software engineering*, no. 4, pp. 279–290, 1977.

[3] R. A. DeMillo, R. J. Lipton, and F. G. Sayward, "Hints on test data selection: Help for the practicing programmer," *Computer*, vol. 11, no. 4, pp. 34–41, 1978.

[4] M. Kintis, M. Papadakis, and A. Papadopoulos, "How effective are mutation testing tools ? an empirical analysis of java mutation testing tools with manual analysis and real faults," *Empirical Software Engineering*, vol. 8, pp. 1–38, 2017.

[5] Y. Jia and M. Harman, "An analysis and survey of the development of mutation testing," *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, 2011.

[6] C. Sun, F. Xue, and H. Liu, "A path-aware approach to mutant reduction in mutation testing," *Information and Software Technology*, vol. 81, pp. 65–81, 2017.

[7] M. Polo, M. Piattini, and I. García-Rodrłguez, "Decreasing the cost of mutation testing with second-order mutants," *Software Testing Verification and Reliability*, vol. 19, no. 2, pp. 111–131, 2009.

[8] M. Jimenez, T. T. Checkam, M. Cordy, M. Papadakis, M. Kintis, Y. L. Traon, and M. Harman, "Are mutants really natural?: a study on how naturalness helps mutant selection," in *Proceedings of the 12th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, 2018, p. 3.

[9] A. Derezińska and A. Rudnik, "Evaluation of mutant sampling criteria in object-oriented mutation testing," in *Computer Science and Information Systems*, 2017, pp. 1315–1324.

[10] D. W. Gong, G. J. Zhang, X. J. Yao, and L. M. Fan, "Mutant reduction based on dominance relation for weak mutation testing," *Information and Software Technology*, vol. 81, pp. 82–96, 2017.

[11] R. Gopinath, I. Ahmed, and M. A. Alipour, "Mutation reduction strategies considered harmful," *IEEE Transactions on Reliability*, vol. 66, no. 3, pp. 854–874, 2017.

[12] A. S. Namin, X. Xue, O. Rosas, and P. Sharma, "Muranker: a mutant ranking tool," *Software Testing Verification and Reliability*, vol. 25, no. 5, pp. 572–604, 2015.

[13] Y. S. Ma and S. W. Kim, "Mutation testing cost reduction by clustering overlapped mutants," *Journal of Systems and Software*, vol. 115, no. 2, pp. 18–30, 2016.

[14] J. Zhang, L. Zhang, M. Harman, D. Hao, Y. Jia, and L. Zhang, "Predictive mutation testing," *IEEE Transactions on Software Engineering*, 2018.

[15] M. Sridharan and A. S. Namin, "Prioritizing mutation operators based on importance sampling," in *IEEE International Symposium on Software Reliability Engineering*, 2010, pp. 378–387.

[16] S. Hussain, "Mutation clustering," Master's thesis, London: King's College, 2008.

[17] Y. H. Huang, "Research on reducing the cost of mutation testing," Master's thesis, Hefei: University of science and technology of china, 2011.

[18] M. Papadakis, Y. L. Traon, O. Rosas, and P. Sharma, "Mutation testing strategies using mutant classification," *Annual ACM Symposium on Applied Computing*, vol. 25, no. 5, pp. 572–604, 2015.

[19] C. Ji, Z. Chen, and B. Xu, "A novel method of mutation clustering based on domain analysis," in *SEKE*, vol. 9, 2009, pp. 422–425.

[20] G. J. Zhang, D. W. Gong, and X. J. Yao, "Test case generation based on mutation analysis and set volution," *Chinese Journal of Computers*, vol. 38, no. 11, pp. 2318–2331, 2015.

[21] F. C. Souza, M. Papadakis, and Y. Le Traon, "Strong mutation-based test data generation using hill climbing," in *Proceedings of the 9th International Workshop on Search-Based Software Testing*, 2016, pp. 45–54.

[22] F. C. Souza, M. Papadakis, V. H. S. Durelli, and M. E. Delamaro, "Test data generation techniques for mutation testing: A systematic mapping," in *Proceedings of the 11th ESELAW*, 2014, pp. 1–14.

[23] Y. Jia and H. M, "Milu: A customizable, runtime-optimized higher order mutation testing tool for the full c language," in *IEEE Practice and Research Techniques.*, 2008, pp. 94–98.

[24] R. A. Silva, S. R. S. Souza, and P. S. L. De-Souza, "A systematic review on search based mutation testing," *Information and Software Technology*, vol. 81, pp. 19–35, 2017.

[25] D. Gong, B. Sun, X. Yao, and T. Tian, "Test data generation for path coverage of mpi programs using saeo," *ACM Transactions on Software Engineering and Methodology*, vol. 30, no. 2, pp. 1–37, 2021.

[26] B. Sun, J. Wang, D. Gong, and T. Tian, "Scheduling sequence selection for generating test data to cover paths of mpi programs," *Information and Software Technology*, vol. 114, pp. 190–203, 2019.

[27] X. J. Yao and D. W. Gong, "Genetic algorithm-based test data generation for multiple paths via individual sharing," *Computational Intelligence and Neuroscience*, vol. 29, pp. 1–13, 2014.

[28] S. Miyamoto, H. Ichihashi, and K. Honda, *Algorithms for Fuzzy Clustering*, 2008.

[29] H. Jiang, X. Chen, and H. T, "Fuzzy clustering of crowdsourced test reports for apps," *ACM Transactions on Internet Technology*, vol. 18, no. 2, pp. 138–153, 2018.

[30] A. Gosain and S. Dahiya, "Performance analysis of various fuzzy clustering algorithms: A review," *Procedia Computer Science*, vol. 79, pp. 100–111, 2016.

[31] X. Dang, X. Yao, D. Gong, and T. Tian, "Efficiently generating test data to kill stubborn mutants by dynamically reducing the search domain," *IEEE Transactions on Reliability*, vol. 69, no. 1, pp. 334–348, 2019.

[32] A. J. Offutt and K. N. King, "A fortran 77 interpreter for mutation analysis," *Acm Sigplan Notices*, vol. 22, no. 7, pp. 177–188, 1987.

[33] X. Yao, M. Harman, and Y. Jia, "A study of equivalent and stubborn mutation operators using human analysis of equivalence," in *Proceedings of the 36th International Conference on Software Engineering*, 2014, pp. 919–930.

[34] M. Kintis, M. Papadakis, and N. Malevris, "Employing second-order mutation for isolating first-order equivalent mutants," *Annual ACM Symposium on Applied Computing*, vol. 25, no. 5, pp. 508–535, 2015.

[35] M. Papadakis and N. Malevris, "Automatically performing weak mutation with the aid of symbolic execution, concolic testing and search-based testing," *Software Quality Journal*, vol. 19, no. 4, pp. 691–723, 2011.

[36] D. Schuler and A. Zeller, "Covering and uncovering equivalent mutants," *Software Testing, Verification and Reliability*, vol. 23, no. 5, pp. 353–374, 2013.

[37] A. J. Offutt and J. Pan, "Automatically detecting equivalent mutants and infeasible paths," *Software testing, verification and reliability*, vol. 7, no. 3, pp. 165–192, 1997.

[38] E. B, "Cluster analysis," *Quality and Quantity*, vol. 14, no. 1, pp. 75–100, 1980.

[39] R. Gelbard, O. Goldman, and I. Spieglcr, "Investigating diversity of clustering methodsan empirical comparison," *Data and Knowledge Engineering*, vol. 63, no. 1, pp. 155–166, 2007.

[40] B. Sun, D. Gong, T. Tian, and X. Yao, "Integrating an ensemble surrogate model's estimation into test data generation," *IEEE Transactions on Software Engineering*, 2020.

[41] M. Masud, A. Nayak, and M. Zaman, "Strategy for mutation testing using genetic algorithms," in *Proc. Canadian Conference on Electrical and Computer Engineering (ICCSCEI)*, 2005, pp. 1049–1052.

[42] M. Papadakis, T. C. Thierry, and L. T. Yves, "Mutant quality indicators," in *Proc. IEEE ICSTW*, 2018, pp. 33–39.

[43] W. Visser, "What makes killing a mutant hard," in *Proc. 31st IEEE/ACM Int. ASE Conf*, 2016, pp. 39–44.

[44] X. Y. Dang, D. W. Gong, and X. J. Yao, "Feasible path generation of weak mutation testing based on statistical analysis," *Chinese Journal of computer science*, vol. 39, no. 11, pp. 2355–2371, 2016.

[45] B. Korel, "Automated software test data generation," *IEEE Transaction on Software Engineering*, vol. 16, no. 8, pp. 870–879, 1990.

[46] S. Mouchawrab, L. C. Briand, Y. Labiche, and M. D. Penta, "Assessing, comparing, and combining state machine-based testing and structural testing: A series of experiments," *IEEE Transactions on Software Engineering*, vol. 37, no. 2, pp. 161–187, 2011.

[47] D. W. Gong, B. Qin, and T. Tian, "Selecting objects to be mutated based on statement importance," *Acta Electronica Sinica*, vol. 45, no. 6, pp. 1518–1522, 2017.

[48] A. C. Barus, T. Y. Chen, F. C. Kuo, H. Liu, and G. Rothermel, "A cost-effective random testing method for programs with non-numeric inputs," *IEEE Transactions on Computers*, vol. 65, pp. 3509–3523, 2016.
[49] H. Liu, F. C. Kuo, D. Towey, and T. Y. Chen, "How effectively does metamorphic testing alleviate the oracle problem ?" *IEEE Transactions on Software Engineering*, vol. 40, no. 1, pp. 4–22, 2014.
[50] V. S, G. M, and D. S, "Goal-oriented mutation testing with focal methods," in *Proc. 9th IACM SIGSOFT International Workshop*, 2018, pp. 39–44.
[51] Z. J. M, Z. L, and H. D, "An empirical comparison of mutant selection assessment metrics," in *IEEE International Conference on Software Testing*.
[52] B. T. F. Chekam T T, Papadakis M, "Selecting fault revealing mutants," *Empirical Software Engineering*, vol. 25, no. 1, pp. 434–487, 2020.
[53] P. L. J. A and V. S. R., "A systematic mapping study on higher order mutation testing," *Journal of Systems and Software*, vol. 154, pp. 92–109, 2019.
[54] J. Strug and B. Strug, "Using classification for cost reduction of applying mutation testing," in *Computer Science and Information Systems*, 2017, pp. 99–108.
[55] I. Gath and A. B. Geva, "Unsupervised optimal fuzzy clustering," *Pattern Analysis & Machine Intelligence IEEE Transactions*, vol. 11, no. 7, pp. 773–780, 1989.
[56] R. A. Demilli and A. J. Offutt, "Constraint-based automatic test data generation," *IEEE Transactions on Software Engineering*, vol. 17, no. 9, pp. 900–910, 2002.
[57] M. Papadakis and N. Malevris, "Automatic mutation test case generation via dynamic symbolic execution," in *Proc. 21st IEEE int.ISSRE Conf.* IEEE, 2010, pp. 121–130.
[58] Y. Zhang and D. W. Gong, "Evolutionary generation of test data for paths coverage based on scarce data capturing," *Chinese Journal of Computers*, vol. 36, no. 36, pp. 2429–2440, 2013.
[59] K. Ayari, S. Bouktif, and G. Antoniol, "Automatic mutation test input data generation via ant colony," in *Proc. Genetic and Evolutionary Computation Conference(GECCO)*, 2007, pp. 1074–1081.
[60] M. Papadakis, N. Malevris, and K. M, "Towards automating the generation of mutation tests," in *Proc. 5th Workshop on Automation of Software Test (AST)*, 2010, pp. 111–118.

Xiangying Dang received the M.S. degree in computer science from the School of Information Engineering, Jiangnan University in 2008. She is currently Ph.D. candidate at the School of Information and Control Engineering, China University of Mining and Technology. Her main research interests include software engineering based Search and Mutation testing and analysis.

Dunwei Gong received the Ph.D. degree in control theory and control engineering from China University of Mining and Technology in 1999. He is a professor in the School of Information and Control Engineering, China University of Mining and Technology. His main research interests include intelligence optimization and control.

Xiangjuan Yao received the Ph.D. degree in control theory and control engineering from China University of Mining and Technology in 2011. She is a professor in the School of Mathematics, China University of Mining and Technology. Her main research interests include intelligence optimization and search based software testing.

Tian Tian received the Ph.D. degree in control theory and control engineering from China University of Mining and Technology, Xuzhou, China, in 2014. She is currently an associate professor in the School of Computer Science and Technology, Shandong Jianzhu University. Her current research interest is parallel program testing.

Huai Liu is a Lecturer in Department of Computer Science and Software Engineering at Swinburne University of Technology, Melbourne, Australia. He has worked as a Lecturer at Victoria University and a Research Fellow at RMIT University. He received the BEng in physioelectronic technology and MEng in communications and information systems, both from Nankai University, China, and the PhD degree in software engineering from the Swinburne University of Technology, Australia. Prior to working in Higher Education, he worked as an engineer in the IT industry. His current research interests include software testing, cloud computing, and end-user software engineering.